Ollama Local AI Chatbot

Project Documentation

An interactive chatbot built with Python, Streamlit and Ollama that runs large language models fully on your own machine.

Item	Details
Author	Surya
GitHub	github.com/suryaantarvedi
Language	Python 3.9+
Framework	Streamlit
LLM runtime	Ollama (local inference)
Date	November 2026

1. Overview

This project is a chat application that lets a user talk to a large language model (LLM) through a clean web interface. Unlike most chatbots, it does not call a paid cloud API. The model runs locally through Ollama, so there are no API keys or usage costs, and conversations never leave the user's computer.

The interface is built with Streamlit, which turns a normal Python script into an interactive web app. The Python code talks to Ollama's local REST API and streams the answer back word by word, so the user sees the response as it is generated.

2. Objectives

- Build a ChatGPT-style chat interface using only Python.
- Run LLMs locally for privacy, zero cost and offline use.
- Give users control over the model, creativity level and assistant behaviour.
- Keep conversation context so follow-up questions work naturally.
- Handle common failures (Ollama not running, model not downloaded) with clear guidance.

3. Technology Stack

Component	Role in the project
Python	Core language for the whole application.
Streamlit	Web UI: chat messages, chat input, sidebar controls, session state.
Requests	Sends HTTP requests to the Ollama API, including streaming reads.
Ollama	Local runtime that downloads, manages and serves the LLMs.
Local LLMs	llama3.2, llama3.1, llama3, mistral, gemma2, qwen2.5, qwen2.5:3b.
HTML / CSS	Custom styling injected into Streamlit for the dark theme and green accents.

4. System Architecture

The system has four parts. The browser shows the interface, the Streamlit app holds the logic, the Ollama server exposes the model over a local HTTP endpoint, and the LLM itself generates the text.

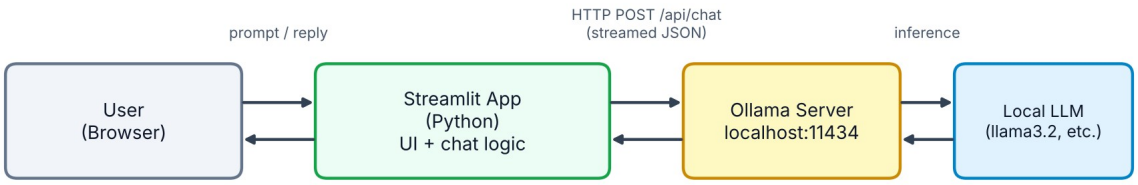


Figure 1: High-level architecture

All communication happens on `localhost`. The Streamlit app sends the conversation to `http://localhost:11434/api/chat` and receives the reply as a stream of small JSON chunks.

5. Application Flow

Streamlit re-runs the script from top to bottom every time the user interacts with the page. The flow below shows what happens in a single run, and how the loop continues for each new message.

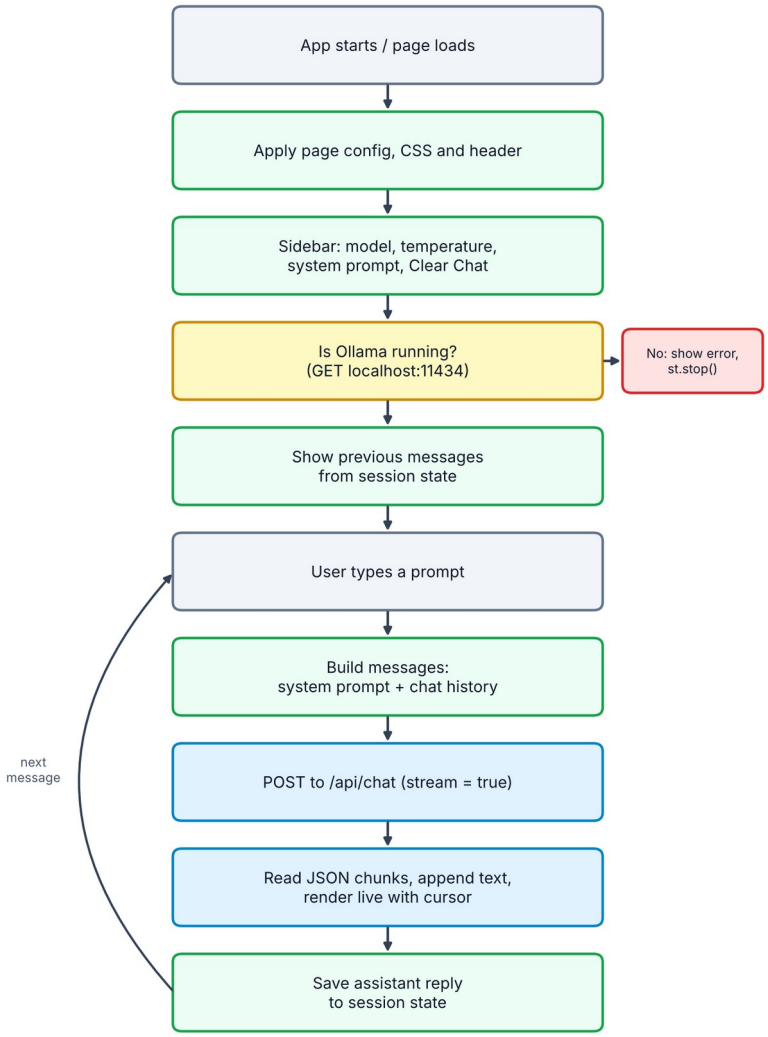


Figure 2: End-to-end application flow

Step-by-step explanation

1. **Page setup.** The app sets the page title, icon and layout, then injects custom CSS and renders the header card.
2. **Sidebar controls.** The user picks a model, sets the temperature, edits the system prompt, or clears the chat.
3. **Health check.** The app calls Ollama's base URL. If there is no response, it shows instructions to start Ollama and stops, so the user never gets a confusing failure later.
4. **Load history.** Earlier messages are read from `st.session_state.messages` and redrawn in the chat window.

5. **User prompt.** When the user submits text, it is shown immediately and appended to the history.
6. **Build request.** The system prompt is placed first, followed by the full conversation history. This is how the model remembers earlier turns.
7. **Stream the reply.** The request is sent with `stream: true`. The app reads the response line by line, parses each JSON chunk, appends its text and updates the screen with a typing cursor until the chunk marked `done` arrives.
8. **Save the answer.** The final text is stored in session state, so the next question includes it as context.

6. Code Walkthrough

Section of the code	What it does
Page Config	set_page_config sets the title, robot icon and centred layout.
Custom CSS	Styles the container width, gradient header card, green accent colour and button hover effect.
Header	Renders the title and subtitle inside an HTML card.
Sidebar settings	Model dropdown, temperature slider (0.0 to 1.0), system prompt text area and Clear Chat button.
check_ollama_running()	GET request to localhost:11434 with a 3 second timeout. Returns True on HTTP 200, otherwise False.
Session state	Creates an empty messages list the first time the app runs and keeps it across re-runs.
Chat history display	Loops through saved messages and renders each with st.chat_message.
Chat input and streaming	Builds the payload, posts to /api/chat with streaming, and updates a placeholder as chunks arrive.
Error handling	HTTPError suggests running ollama pull for the missing model. Any other exception is shown as a general error.

Request payload

Each message sends a payload with this structure:

```
{  "model": "llama3.2",
   "messages": [system prompt, ...chat history],
   "stream": true,
   "options": { "temperature": 0.7 }  }
```

7. Key Features

- **Streaming responses:** text appears token by token with a typing cursor.
- **Conversation memory:** full history is sent each turn within a session.
- **Model selection:** switch between seven local models from the sidebar.
- **Temperature control:** low values give focused answers, high values give more creative ones.
- **Editable system prompt:** change the assistant's role and tone without touching the code.
- **Clear Chat:** resets the conversation in one click.

- **Health check and errors:** clear messages for a stopped server or a missing model.
- **Privacy:** everything runs locally, nothing is sent to external servers.

8. Setup and Run

Prerequisites

- Python 3.9 or higher.
- Ollama installed from ollama.com/download.

Installation

1. `pip install streamlit requests`
2. Start Ollama: `ollama serve`
3. Download a model: `ollama pull llama3.2`
4. Run the app: `streamlit run app.py`
5. Open `http://localhost:8501` in the browser.

9. Configuration

Setting	Location	Default
Ollama URL	OLLAMA_URL in the code	http://localhost:11434/api/chat
Model list	Sidebar selectbox	llama3.2 and six others
Temperature	Sidebar slider	0.7
System prompt	Sidebar text area	Helpful assistant that explains clearly
Request timeout	requests.post call	120 seconds

10. Troubleshooting

Problem	Solution
Ollama is not running error	Run ollama serve in a terminal and refresh the page.
HTTP error or model not found	Run ollama pull with the model name selected in the sidebar.
Slow responses	Use a smaller model such as llama3.2 or qwen2.5:3b.
Request timed out	Increase the timeout value or switch to a lighter model.
Chat disappears on refresh	Expected behaviour. History lives in session state, not on disk.

11. Limitations and Future Work

Current limitations

- Chat history is lost when the browser tab is refreshed or closed.
- Very long chats may exceed the model's context window because the full history is sent every turn.
- Ollama must run on the same machine unless the URL is changed.

Planned improvements

- Save and load conversations using JSON or SQLite.
- Detect installed models automatically using Ollama's /api/tags endpoint.
- Export a conversation as text or PDF.
- Document question answering (RAG) with local embeddings.
- Docker setup for one-command deployment.

12. Conclusion

The project shows how a complete, private chatbot can be built with a few dozen lines of Python. It combines a modern chat UI, real-time streaming, adjustable model settings and sensible error handling, and it forms a solid base for features such as document Q&A, saved chats or a custom assistant.